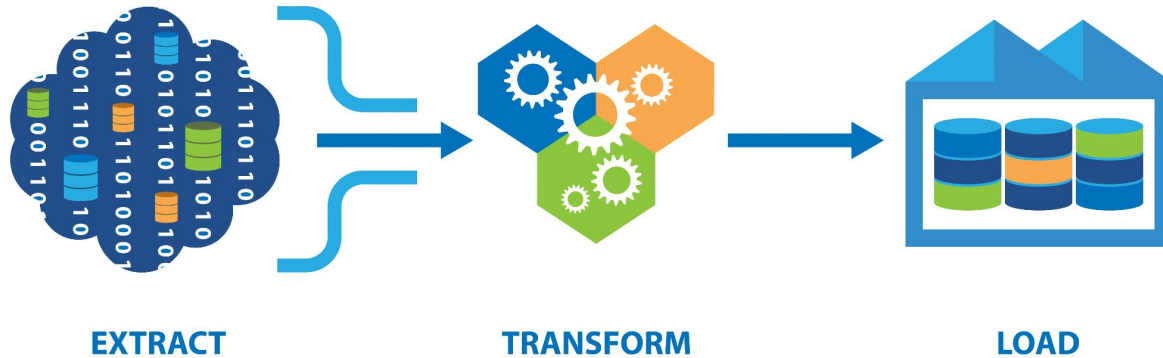


ETL Process

Dr. Moamin Abughazala

What is ETL?

ETL stands for extract, transform, and load. It is a process that is used to get the data from a variety of sources, clean and transform the data and then load it into a destination such as a data warehouse or a data mart.



Example

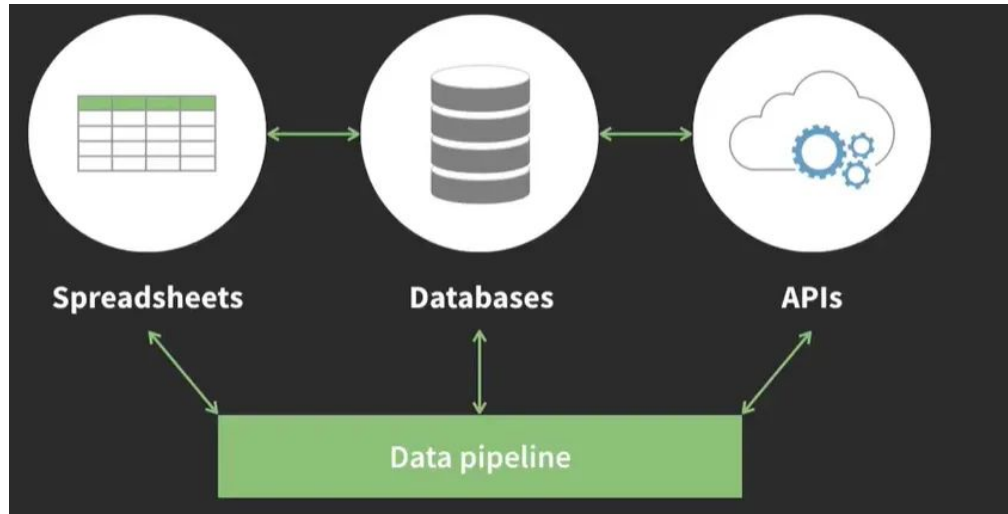
- Let's take the insurance industry as an example. An insurance company may have policyholder information stored in different system, such as their website, call center, and agents. The ETL process would extract the data from these different systems, clean and transform the data and then load into our data warehouse.
- This would give the company a single view of policyholder information which can be used for reporting and analytics

Another Example

- A bank may have customer information stored in different systems, such as an online banking which could be one database.
- ATM transaction might have another database and branches might have their own separate database.
- The ETL process would extract the data from these different systems, clean and transform the data and then load it into a data warehouse.
- This will give the bank a single view of customer information which could be used for reporting and analytics, such as identifying patterns and trends in the customer behavior.

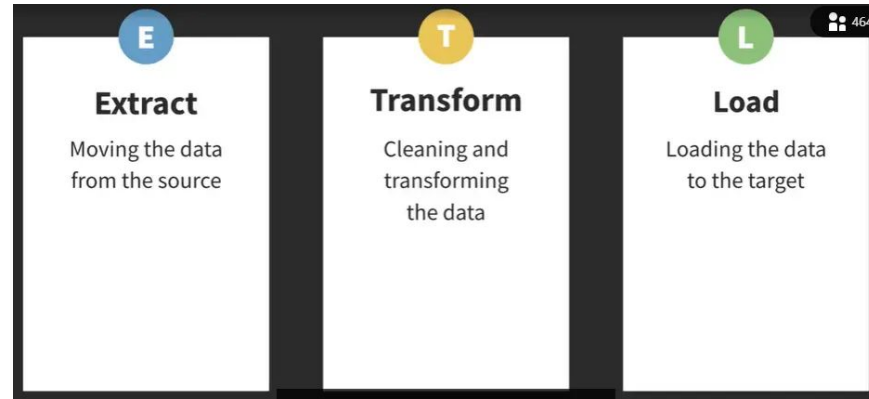
Data sources

- In many companies and organizations, data is spread across different sources, like spreadsheets, production databases, or APIs.
- A data pipeline helps combine data from these different sources into one consistent and unified format to support analysis and other business needs.



What is data Pipeline ?

- A data pipeline is a set of **processes** and technologies that describe a data's journey or movement from the data source to its final destination
- It is very essential in the data ecosystem
- ETL stands for extract, transform, and load. (.It is a very specific type of data pipeline)



1. The **extract** in ETL involves extracting data from various sources, systems, or applications, like spreadsheets, APIs, databases, and cloud systems, in preparation for the transformation phase. The extract step can be done using common tools like Python, AWS Glue, Azure Data Factory, Talend, and many more
2. The **transform** step in ETL involves cleaning, enriching, aggregating, and standardizing the data to remove errors, duplicates, or irrelevant information in the data. It is then converted to a useful format for analysis. In this step, we ensure that the data is **enriched, aggregated, and consistent**. Common tools used in the data transformation steps include Python, Spark, and Azure Data Factory.
3. The **loading** process involves loading data that has been transformed into target databases, usually a **data warehouse**.
 - a. The end users, it could be data scientists, data analysts, business analysts, can assess the data to create **reports, dashboards** build machine learning models, the list goes on and on. Common tools used to load data into target systems include Python, Spark, Azure Data Factory, amongst others

Environment Setup

Visual Studio Code + Jupyter

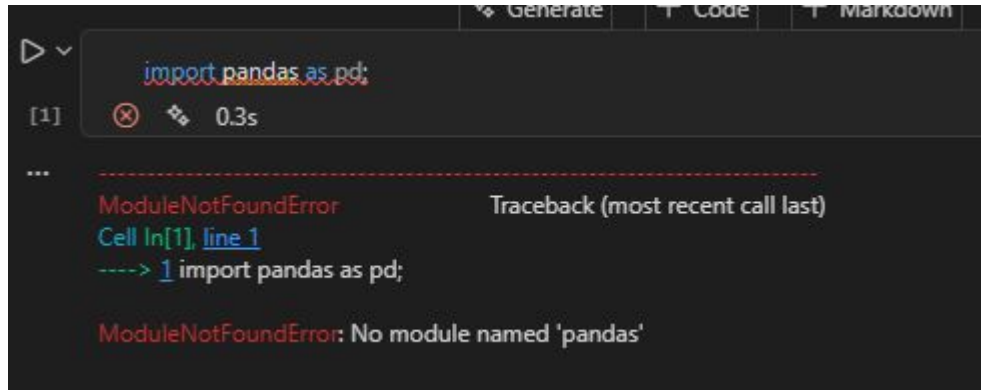
1. Install VS Code
2. Install Python
3. Install Jupyter
 - a. `pip install notebook jupyterlab` or
 - b. `python -m pip install notebook jupyterlab`
 - c. `python -m pip install pandas`
4. Install VS Code Extensions
 - a. Python (by Microsoft)
 - b. Jupyter (by Microsoft)
5. Or `python -m jupyter lab`

1. Set Up Python Environment

1. Ctrl+Shift+P

1. Python: Select Interpreter

python -m pip install pandas



The screenshot shows a Jupyter Notebook interface with a dark theme. At the top, there are tabs for 'Generate', 'Code', and 'Markdown'. Below the tabs is a code editor with the text `import pandas as pd;`. Below the code editor is a cell output area. It starts with a red 'x' icon, a red 'x' icon, and the text '0.3s'. Below this is a red dashed line, followed by the text 'ModuleNotFoundError' and 'Traceback (most recent call last)'. Below this is the text 'Cell In[1], line 1' and '----> 1 import pandas as pd;'. Below this is the text 'ModuleNotFoundError: No module named 'pandas''.

```
import pandas as pd;
```

[1] 0.3s

ModuleNotFoundError Traceback (most recent call last)

Cell In[1], line 1

----> 1 import pandas as pd;

ModuleNotFoundError: No module named 'pandas'

What is Pandas?

Pandas is a popular open source Python library used for **data manipulation and analysis**. It provides easy-to-use functions and methods for working with data.

```
import pandas as pd;
```

Data structures

They are **series and dataframes**.

1. **Series** are like **one dimensional arrays** with **label data**, sort of like a **column** in an **Excel spreadsheet**.

```
s = pd.Series([10, 20, 30], index=['A', 'B', 'C'])
```

```
print(s)
```

A 10
B 20
C 30

Index
(labels) Values

x	1
y	2
z	3

dtype: int64

Pandas Series: Values and labels

```
import pandas as pd;
```

✓ 0.0s

```
data= pd.Series([10,20,30,40,50]);
```

✓ 0.0s

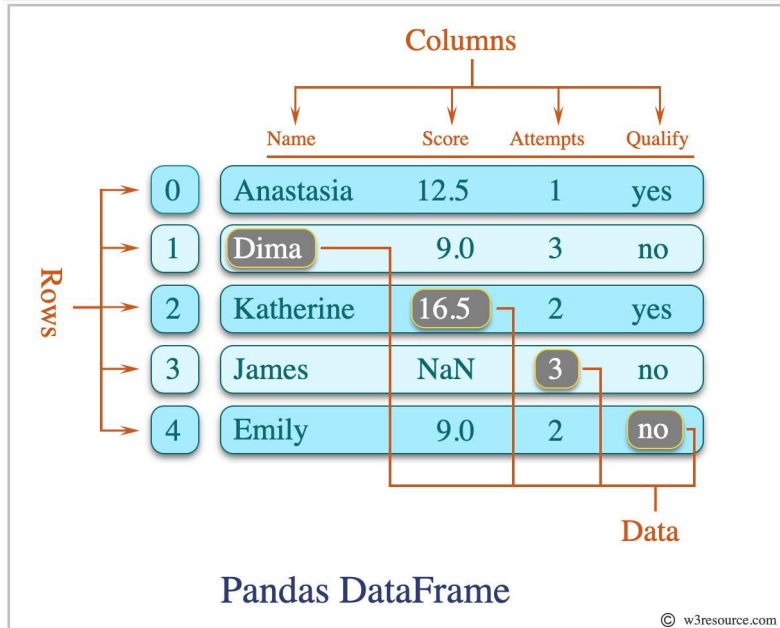
data

✓ 0.0s

0 10
1 20
2 30
3 40

Data structures

- **Dataframes** are like **two dimensional tables with rows x columns**, like an Excel spreadsheet or an SQL table. Dataframes allow you the flexibility to store and manipulate data in a structured format.



```
data={
    'Name':['Ali','Sami','Sarah','Ahamd'],
    'Age':[25,40,50,44],
    'City':['Nablus', 'Jenin', 'Tulkarem', 'Rome']
}
```

[1]

```
df=pd.DataFrame(data);
```

[8]

✓ 0.0s



```
df
```

[9]

✓ 0.0s

Understanding the Data using Panda

means exploring, summarizing, and checking your dataset before doing any analysis or modeling.

This is often called Exploratory Data Analysis (EDA)

Understanding the Data using Panda

- Understanding the data is a big part of data warehousing and ETL creation.
- There are a couple of reasons why we need to explore and understand our data before transformation and warehousing. Before warehousing our data, we must be familiar with the data structure, format, data types, and relationships between tables and columns.
- This is very important for the transformation stage of the ETL. We also need to consider data quality. Are there **duplicates, inconsistencies, missing values, or no records?**
 - a. To address any data quality issues.
 - b. To Know what transformations are required.
 - c. Which business logics that should be applied to the data.
 - d. We also need to figure out how to structure the ETL process as well as what columns, if any, need to be standardized. For example, converting time columns from string to daytime if necessary or converting from a 12-hour data format to an 18-four-hour data format.

Explore the data using Pandas

- **df.head()** : used to display the **first few rows** of a DataFrame. By default, it returns **the first 5 rows**, but you can specify a different number by passing an integer. (df.head(3))
- **df.tail()** : It returns the **last few rows** of a DataFrame. By default, it shows the **last 5 rows**, but you can specify a different number.
- Use this data :

```
data = {  
    "id": [1, 2, 3, 4, 5],  
    "product customer purchased": ["Widget A", "Widget B", "Widget A", "Widget C", "Widget B"],  
    "Name of customer": ["John Doe", "Jane Smith", "Bob Johnson", "Susan Brown", "Mary Wilson"],  
    "address of customer": ["123 Main St", "456 Elm St", "789 Oak St", "234 Pine St", "567 Birch St"],  
    "email of customer": ["john.doe@email.com", "", "bob.johnson@email.com", "susan.brown@email.com", "mary.wilson@email.com"],  
    "phone number of customer": ["123-456-7890", "987-654-3210", "555-123-7890", "777-888-9999", "111-222-3333"],  
    "time of purchase": [  
        "2023-08-10 09:15:00",  
        "2023-08-10 10:30:00",  
        "2023-08-11 14:45:00",  
        "2023-08-11 15:30:00",  
        "2023-08-12 11:20:00"  
    ]  
}
```

```
df=pd.DataFrame(data)
```

```
df.head()
```

	id	product customer purchased	Name of customer	address of customer	email of customer	phone number of customer	time of purchase
0	1	Widget A	John Doe	123 Main St	john.doe@email.com	123-456-7890	2023-08-10 09:15:00
1	2	Widget B	Jane Smith	456 Elm St		987-654-3210	2023-08-10 10:30:00
2	3	Widget A	Bob Johnson	789 Oak St	bob.johnson@email.com	555-123-7890	2023-08-11 14:45:00
3	4	Widget C	Susan Brown	234 Pine St	susan.brown@email.com	777-888-9999	2023-08-11 15:30:00
4	5	Widget B	Mary Wilson	567 Birch St	mary.wilson@email.com	111-222-3333	2023-08-12 11:20:00

```
df.tail()
```

	id	product customer purchased	Name of customer	address of customer	email of customer	phone number of customer	time of purchase
0	1	Widget A	John Doe	123 Main St	john.doe@email.com	123-456-7890	2023-08-10 09:15:00
1	2	Widget B	Jane Smith	456 Elm St		987-654-3210	2023-08-10 10:30:00
2	3	Widget A	Bob Johnson	789 Oak St	bob.johnson@email.com	555-123-7890	2023-08-11 14:45:00
3	4	Widget C	Susan Brown	234 Pine St	susan.brown@email.com	777-888-9999	2023-08-11 15:30:00
4	5	Widget B	Mary Wilson	567 Birch St	mary.wilson@email.com	111-222-3333	2023-08-12 11:20:00

Df.shape: attribute in pandas returns a tuple representing the dimensions of a DataFrame: (number_of_rows, number_of_columns). a tuple representing the dimensions of a DataFrame. **(5, 7)**

The **df.info()** method provides a concise summary of a DataFrame, which is very useful for inspecting the structure and memory usage of your data.

What it shows:

- The number of non-null entries per column
- Column names and data types (dtype)
- Index range
- Total memory usage

Show DataFrame info

df.info()

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 5 entries, 0 to 4
```

```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	id	5 non-null	int64
1	product customer purchased	5 non-null	object
2	Name of customer	5 non-null	object
3	address of customer	5 non-null	object
4	email of customer	5 non-null	object
5	phone number of customer	5 non-null	object
6	time of purchase	5 non-null	object

```
dtypes: int64(1), object(6)
```

```
memory usage: 412.0+ bytes
```

Show the duplicated rows

The `df.duplicated()` method in pandas is used to identify **duplicate rows** in a DataFrame. It returns a Boolean Series where:

- True indicates a duplicate row
- False indicates a unique row (or the first occurrence of a duplicate)

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Alice', 'David'],
        'Age': [25, 30, 25, 40]}

df = pd.DataFrame(data)

print(df.duplicated())
```

[10] ✓ 0.0s

```
... 0    False
    1    False
    2     True
    3    False
    dtype: bool
```

Remove duplicates

`.drop_duplicates()` removes all duplicate rows, keeping only the first occurrence

`inplace=True` updates the **original** DataFrame instead of returning a new one

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Alice', 'David'],
        'Age': [25, 30, 25, 40]}

df = pd.DataFrame(data)

df.duplicated()
df[df.duplicated()]          # View only duplicate rows
#Result: A new DataFrame showing only the duplicate rows (excluding the first occurrence).

df.drop_duplicates(inplace=True) # Remove duplicates , inplace=True updates the original DataFrame instead of returning a new one
#To remove duplicates based on a specific column, use the subset parameter
df.drop_duplicates(subset=['Name'], inplace=True)
df.duplicated()
```

Extract the Data

Data Sources

Organizations have many data sources available :

- SQL or a NoSQL database
 - APIs
 - JSON Files
 - Excel or CSV
-
- Data also exists in **different formats** like XML, Parquet, JSON, and TXT

Loading data from CSV

```
df = pd.read_csv()
```

```
df = pd.read_csv('data.csv', sep=';')
```

Optional parameters:

- `sep=',':` Specify delimiter (e.g., `;` for semicolon-separated files)
- `header=0:` Use first row as column names
- `nrows=10:` Read only first 10 rows
- `usecols=['col1', 'col2']:` Read only selected columns

Loading data from EXCEL File

```
df = pd.read_excel('data.xlsx')
```

```
df = pd.read_excel('report.xlsx', sheet_name='SalesData')
```

- `sheet_name=0`: Read the first sheet
- `sheet_name='Sheet2'`: Read a specific sheet by name
- `usecols="A:C"`: Read specific columns by Excel letters or names
- `nrows=5`: Read only first 5 rows

Extract data from a MySQL table into a pandas

DataFrame

pip install sqlalchemy

```
import pandas as pd
from sqlalchemy import create_engine

# Step 1: Create connection string
username = 'root'
password = 'root'
host = '127.0.0.1'
port = 3306 # Default MySQL port
database = 'sakila'

# Format: "mysql+pymysql://username:password@host:port/database"
connection_string = f"mysql+pymysql://{username}:{password}@{host}:{port}/{database}"

# Step 2: Create engine
engine = create_engine(connection_string)

# Step 3: Write a SQL query (e.g., filter & select specific columns)
query = """
SELECT * FROM payment limit 50
"""

# Step 4: Execute the query and load data into a DataFrame
df = pd.read_sql(query, con=engine)

# Step 5: Preview the DataFrame
df.head()
```

Transforming Data

Transforming Data

- Data transformation usually involves **cleaning, standardizing, and removing duplicates and missing values**.
- Data transformation focuses on converting raw data coming from different systems into a format suitable for the target system.
- Data transformation is important for **data quality and consistency**, as inaccurate data can lead to faulty reporting, which can cause an organization to lose money.

Transformation

- Deduplication is simply identifying and **removing duplicates** from your datasets. Data deduplication removes duplicated data before inserting it into the data warehouse.
- **Aggregating** the data allows the company to analyze overall sales performance, identifying top-performing sales teams, popular products, and overall revenue trends.
- Data aggregation is a process of **summarizing, grouping, or consolidating** data from multiple rows or records into a single value or set of values.
- It reduces large amounts of data to more manageable and meaningful representations.

Data filtering enables you to remove unwanted data from an entire dataset. You can choose specific data elements and remove others.

Data cleaning identify and eliminate errors and inconsistencies in our data

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Alice', 'David', 'Eva', 'Frank', None],
    'Age': [25, 30, 25, None, 45, 50, 40],
    'Department': ['HR', 'IT', 'HR', 'IT', 'Finance', 'Finance', 'HR'],
    'Salary': [50000, 60000, 50000, 70000, 80000, 80000, None]
}

df = pd.DataFrame(data)

print("Original DataFrame:\n", df)
```

Remove Duplicates

```
df_no_duplicates = df.drop_duplicates()  
print("\nAfter removing duplicates:\n", df_no_duplicates)
```

//You can also remove duplicates based on specific columns:

```
df_unique_names = df.drop_duplicates(subset='Name', keep='first')
```

Cleaning: Handle Missing Values

Drop rows with any missing values, This line removes all rows from the DataFrame df that contain at least one missing value (NaN).

```
df_cleaned = df.dropna()
```

```
print("\nAfter dropping rows with missing values:\n", df_cleaned)
```

//Creates a copy of the original DataFrame df and assigns it to df_filled, so the original data remains unchanged while filling values.

```
df_filled = df.copy()
```

Fill missing values (e.g., fill Age with mean)

```
df_filled['Age'].fillna(df['Age'].mean(), inplace=True)//Fills the missing values in the Age column with the mean of all existing (non-null) Age values.
```

```
df_filled['Salary'].fillna(0, inplace=True)// Fills missing values in the Salary column with 0
```

```
df_filled['Name'].fillna("Unknown", inplace=True)// Fills missing values in the Name column with the string "Unknown".
```

```
print("\nAfter filling missing values:\n", df_filled)
```

Filtering Rows

```
# Average salary by department
```

```
avg_salary = df.groupby('Department')['Salary'].mean().reset_index()
```

```
print("\nAverage Salary by Department:\n", avg_salary)
```

```
# Count employees per department
```

```
count_by_dept =
```

```
df.groupby('Department')['Name'].count().reset_index(name='EmployeeCount')
```

```
print("\nEmployee Count by Department:\n", count_by_dept)
```

```
avg_salary = df.groupby('Department')['Salary'].mean().reset_index()
```

- `df.groupby('Department')`
 - Groups the DataFrame `df` by unique values in the 'Department' column.
 - Each group contains rows corresponding to a particular department.
- `['Salary']`
 - Selects the Salary column from each department group.
- `.mean()`
 - Calculates the mean (average) salary for each department.
- `.reset_index()`
 - Converts the result from a Series with the department as the index into a DataFrame with a default integer index.
 - This makes it easier to work with the result as a regular DataFrame.

Load Data To SQL

Load To SQL

`df.to_sql(name=table_name, con=engine, if_exists='replace', index=False)`

- The `df.to_sql()` function in Pandas is used to write records stored in a DataFrame to a relational database table. This is commonly used when working with SQL databases like MySQL, PostgreSQL, SQLite, etc.
 - `df` => The DataFrame you want to store in the SQL database.
 - `name=table_name` => The name of the table in the SQL database. If the table does not exist, it will be **created**.
 - `con=engine` => The SQLAlchemy engine or DBAPI2 connection that connects Python to your database
 - `if_exists='replace'` => What to do if the table already exists
 - Fail: Raise an error,
 - Replace: Drop the table and create a new one
 - Append : Add the data to the existing table
 - `index=False` => Whether to write the DataFrame index as a column in the SQL table. False means the index will not be written. Set it to True if you want to preserve the index

